

What changed today

A checked, partner-ready record of the backend API review, final frontend, verification results, and submission handoff.

CHRONOS-H	23 AUGUST 2026
-----------	----------------

Outcome	Chronos is now a complete local demonstration platform: durable Python data engine, REST API, final responsive frontend, benchmark harness, and reproducible verification.
---------	--

Today's scope

Area	Delivered
Backend	Dependency-free REST/JSON workflows over transactional SQLite repositories.
Frontend	End-to-end repository, import, commit, history, checkout, compare, and metrics workspace.
Quality	40 Python tests, production frontend build, lint, and rendered-page tests all passing.
Documentation	API guide, frontend runbook, change record, and separate architecture reference.

1. Backend completed and rechecked

The backend was reviewed against every operation required by the final interface. No required frontend endpoint is missing.

Repository lifecycle: list, create, open, validate repository names, and keep database paths inside the configured root.

Dataset intake: commit keyed JSON state, JSON row arrays using a primary key, or raw UTF-8 CSV without adding a web-framework dependency.

Atomic mutation batches: apply puts and deletes in one SQLite transaction and reject stale base versions with HTTP 409.

Version navigation: newest-first history, paginated checkout, structured comparison, and measured storage/diff metrics.

Safety boundary: CORS allowlist, structured input limits, per-repository write locks, integrity verification, and loopback binding by default.

Audit result

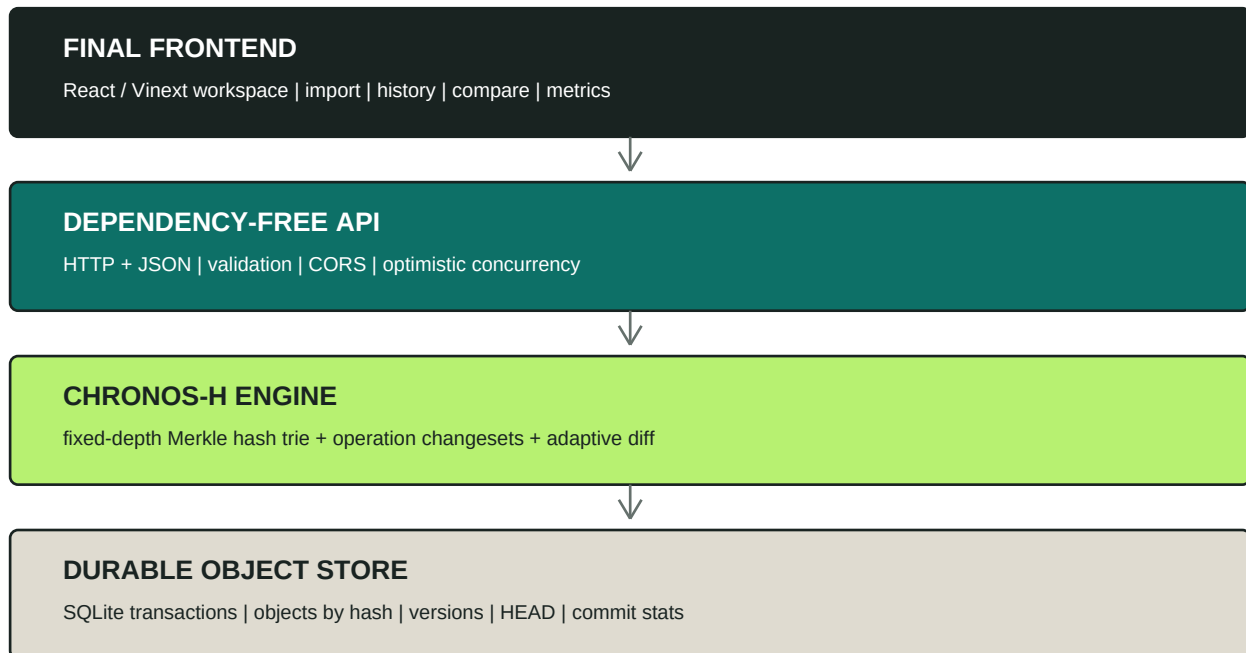
All seven final UI capabilities have a tested backend contract. Authentication, TLS, rate limiting, and multi-host locking remain intentional future-production controls; they are not needed for the local final-submission demo.

Verified API surface

Workflow	Endpoint
Create / list / open	POST + GET /api/repositories; POST /{name}/open
Import / commit	POST /{name}/import; POST /{name}/commits
History / checkout	GET /{name}/history; GET /{name}/versions/{version}
Compare / metrics	GET /{name}/compare; GET /{name}/metrics

2. Final frontend delivered

The frontend is a focused research-system workspace rather than a generic admin dashboard. Its screens expose the specific behavior that distinguishes Chronos.



Screen capability	What the user can prove
Import	Create v1 from CSV or JSON through the same durable commit pipeline.
Atomic commit	Apply multiple puts/deletes with an explicit base version.
History + checkout	Inspect commit ancestry and reproduce historical state.
Version compare	Run Chronos-H, forced Merkle, or forced Log and inspect exact changes.
Metrics + integrity	See new/reused nodes, sharing, diff work, timing, hashes, and verified objects.

Product finish

Responsive desktop/mobile layout; explicit loading, empty, success, failure, and API-offline states; keyboard-visible focus; reduced-motion behavior; typed API client; reconnectable API URL; Open Graph/Twitter metadata; and a Chronos-specific social preview.

3. Verification and handoff

The same code recorded here was built and tested before document generation.

Check	Result	Evidence
Python unit + HTTP suite	PASS	40 / 40 tests
Frontend lint	PASS	0 errors
Production frontend build	PASS	Vinext five-stage build completed
Rendered product tests	PASS	2 / 2 tests
Backend contract audit	PASS	Every final UI action mapped to a route

Run the complete demo

Terminal 1	From the Chronos root: <code>python -m chronos_api</code>
Terminal 2	From frontend: <code>npm install</code> , then <code>npm run dev</code> . Open <code>http://localhost:3000</code> .

What remains for the final paper

- Install/verify Dolt and execute the identical fixed-seed evaluation workloads; never fabricate unavailable Dolt results.
- Run repeated measured trials for Snapshot, Log-only, forced-Merkle, Chronos-H, and Dolt; retain raw CSV, workload manifests, environment metadata, and correctness digests.
- Generate paper figures and statistical summaries from those raw files, then write results, threats to validity, and the literature review around the measured evidence.
- Record the final machine/software configuration and freeze the exact benchmark commit used for every table.

Submission position	The implementation and evaluation infrastructure are ready. The major remaining research task is the real Dolt comparison and the paper built from those measurements.
----------------------------	--